

Design NUFI Personal Finance Management System

Database Management Systems

Le Thi Thuy Trang

Student ID: 11245940

Class: DSEB66B

Faculty of Mathematics and Economics

College of Technology

National Economics University

Table of Contents

- 1 Problem and Requirements
- 2 Database Design
- 3 SQL Implementation
- 4 Web Integration and Demo
- 5 Deployment Extension
- 6 Testing and Evaluation
- 7 Conclusion

Table of Contents

- 1 Problem and Requirements
- 2 Database Design
- 3 SQL Implementation
- 4 Web Integration and Demo
- 5 Deployment Extension
- 6 Testing and Evaluation
- 7 Conclusion

Problem Context

Real-life financial management problem

Personal finance data is generated every day through income, expenses, bank accounts, digital wallets, budgets, debts, and shared transactions. Without a structured database, these records become fragmented and hard to summarize accurately.

- ▶ Manual notebooks or spreadsheets are easy to forget and duplicate.
- ▶ Monthly summaries require repeated manual calculation.
- ▶ Budget tracking and debt status are difficult to keep consistent.
- ▶ Financial data requires privacy and clear user ownership.

Why This Is A DBMS Problem

Data requirements

- ▶ Structured financial records
- ▶ Relationships across tables
- ▶ Time-based reporting
- ▶ User-level data isolation

DBMS responsibilities

- ▶ Enforce primary and foreign keys
- ▶ Validate business constraints
- ▶ Support efficient aggregation
- ▶ Protect consistency across workflows

Project Scope

Core scope

The project focuses on designing, implementing, and validating a MySQL relational database for a personal finance management system.

- ▶ Users, categories, bank accounts
- ▶ Income and expense transactions
- ▶ Budgets, alerts, debts
- ▶ Django integration
- ▶ Web demo for validation
- ▶ Docker and Kubernetes extension

Actors and Main Functions

Guest

- ▶ View landing page
- ▶ Register account
- ▶ Login

User

- ▶ Manage own finance
- ▶ Track budgets
- ▶ View reports

Admin

- ▶ Manage operation
- ▶ Monitor system
- ▶ Support backup

Table of Contents

- 1 Problem and Requirements
- 2 Database Design
- 3 SQL Implementation
- 4 Web Integration and Demo
- 5 Deployment Extension
- 6 Testing and Evaluation
- 7 Conclusion

Conceptual Data Model

Design idea

The user is the central entity. All financial data must be linked to a user or to a controlled sharing group, so ownership and privacy can be enforced consistently.

- ▶ Users own transactions
- ▶ Categories classify cashflow
- ▶ Bank accounts track balances
- ▶ Budgets control spending
- ▶ Debts track obligations
- ▶ Groups enable sharing

Main Entity Groups

Core data

- ▶ users
- ▶ categories
- ▶ bank_accounts
- ▶ incomes
- ▶ expenses

Control and sharing

- ▶ budgets, alerts
- ▶ debts, debt_payments
- ▶ groups, group_members
- ▶ shared_transactions

Relationships

Relationship design

Most relationships are one-to-many and are implemented with foreign keys. Many-to-many relationships are normalized through junction tables.

- ▶ One user can have many incomes, expenses, accounts, budgets, debts.
- ▶ One debt can have many debt payments.
- ▶ One group can have many members.
- ▶ User-group membership is stored in `group_members`.
- ▶ Shared transactions are stored in `shared_transactions`.

Normalization

1NF

Every column stores atomic values. For example, group members are not stored as a list inside groups.

2NF

Non-key attributes depend on the whole primary key. Payment history is separated from debts.

3NF

Non-key attributes do not depend on other non-key attributes. Category names stay in categories.

Keys and Constraints

Keys

- ▶ Primary keys identify records.
- ▶ Foreign keys preserve relationships.
- ▶ Unique keys prevent duplicated accounts.

Constraints

- ▶ Amount values must be positive.
- ▶ Status fields use valid values.
- ▶ Required fields cannot be null.

Index Design

Purpose

Indexes are designed around common access patterns: user filtering, date filtering, category grouping, status filtering, and dashboard aggregation.

- ▶ `idx_incomes_user_income_date`
- ▶ `idx_expenses_user_expense_date`
- ▶ `idx_expenses_user_category_expense_date`
- ▶ `idx_alerts_user_is_read_created_at`
- ▶ `idx_debts_user_status_due_date`

Table of Contents

- 1 Problem and Requirements
- 2 Database Design
- 3 SQL Implementation**
- 4 Web Integration and Demo
- 5 Deployment Extension
- 6 Testing and Evaluation
- 7 Conclusion

SQL File Organization

Implementation structure

SQL scripts are separated by responsibility so the database can be recreated consistently and each DBMS object group is easy to inspect.

- ▶ `schema.sql`
- ▶ `functions.sql`
- ▶ `triggers.sql`
- ▶ `views.sql`
- ▶ `procedures.sql`
- ▶ `indexes.sql`
- ▶ `sample_data.sql`
- ▶ MySQL 8.x runtime

Execution Order

Why order matters

Tables must exist before triggers, views, functions, and procedures can reference them. The project therefore uses a controlled execution order.

1. Create schema and base tables.
2. Create functions used by reports.
3. Create triggers for validation and synchronization.
4. Create views for dashboard and summaries.
5. Create procedures for multi-step workflows.
6. Add indexes and sample data.

Views and Functions

Views

Views are used for reporting and dashboard aggregation, such as monthly expense summary, budget usage, dashboard snapshot, and debt summary.

Functions

Functions centralize reusable formulas, including monthly income total, monthly expense total, monthly net amount, remaining budget, and debt remaining amount.

Procedures and Triggers

Procedures

Stored procedures wrap multi-step operations such as creating expenses, creating incomes, creating debt payments, and generating monthly summaries.

Triggers

Triggers validate inserted or updated data and synchronize derived values such as account balance and debt status.

Trigger Example: Expense

Before insert or update

The trigger checks whether the category is valid, whether the bank account belongs to the correct user, and whether the transaction data is valid.

After insert or update

The trigger updates the related account balance so reports and dashboard values immediately reflect the latest transaction.

Database vs Django Logic

Database layer

- ▶ Enforces PK, FK, constraints
- ▶ Runs views and functions
- ▶ Protects data consistency

Django layer

- ▶ Handles request user context
- ▶ Renders forms and messages
- ▶ Controls page-level permissions

Table of Contents

- 1 Problem and Requirements
- 2 Database Design
- 3 SQL Implementation
- 4 Web Integration and Demo**
- 5 Deployment Extension
- 6 Testing and Evaluation
- 7 Conclusion

Django and MySQL Integration

Integration idea

Django models are mapped to MySQL tables. User actions in the web interface become database operations, and dashboard or report screens read aggregated data from MySQL.

- ▶ UserProfile maps to users.
- ▶ Expense maps to expenses.
- ▶ Budget maps to budgets.
- ▶ Alert maps to alerts.
- ▶ Reports read transaction and budget data.

Demo Position in Presentation

Why demo here?

The demo is placed after database design and SQL implementation, because its purpose is to prove that the DBMS design works in a real application.

1. Open dashboard and observe aggregated values.
2. Add a new expense from the web form.
3. Check that account balance changes.
4. Check budget warning or alert.
5. Open report and verify updated totals.

Demo Script

Main point to explain during demo

One web action is not just one screen update. It creates or changes records in MySQL, triggers validation, updates balances, affects budgets, and changes report results.

- ▶ Dashboard proves aggregation.
- ▶ Expense form proves write workflow.
- ▶ Alert screen proves business rule checking.
- ▶ Report page proves database summarization.

Table of Contents

- 1 Problem and Requirements
- 2 Database Design
- 3 SQL Implementation
- 4 Web Integration and Demo
- 5 Deployment Extension**
- 6 Testing and Evaluation
- 7 Conclusion

Why Deployment Is Included

Position of this section

Deployment is not the main DBMS theory part. It is included to prove that the database has been integrated into a complete runnable system.

- ▶ Local Django with MySQL
- ▶ Docker image
- ▶ Docker Compose stack
- ▶ Kubernetes deployment
- ▶ Helm configuration
- ▶ Cloudflare Tunnel demo

Operations Extension

Automation

- ▶ CronJob for daily summary
- ▶ Budget alert checking
- ▶ Debt overdue checking
- ▶ Smoke testing

Reliability

- ▶ Backup and export
- ▶ Health checks
- ▶ Monitoring metrics
- ▶ Operational logs

Monitoring Stack

Monitoring purpose

Monitoring is added to show that the application and database-backed workflows can be observed during operation.

- ▶ Prometheus collects metrics.
- ▶ Grafana visualizes dashboards.
- ▶ Alertmanager handles technical alerts.
- ▶ Health endpoints support smoke tests.
- ▶ Logs help debug deployment issues.

Table of Contents

- 1 Problem and Requirements
- 2 Database Design
- 3 SQL Implementation
- 4 Web Integration and Demo
- 5 Deployment Extension
- 6 Testing and Evaluation**
- 7 Conclusion

Testing Scope

Functional tests

- ▶ Create expense
- ▶ Update balance
- ▶ Check ownership
- ▶ Create debt payment
- ▶ Generate report

SQL object tests

- ▶ Function returns net amount
- ▶ View returns budget usage
- ▶ Procedure returns summary
- ▶ Trigger rejects invalid alert

Testing Result

Test environment

The test suite was executed on a real MySQL database instead of using SQLite only, so triggers, views, functions, and procedures were validated in the actual DBMS environment.

- ▶ 22 automated tests passed.
- ▶ Ownership protection was verified.
- ▶ SQL objects were loaded and called directly.
- ▶ Business workflows produced expected results.

Evaluation

Database strengths

- ▶ Clear relational design
- ▶ Normalized schema
- ▶ Strong constraints
- ▶ Useful SQL objects

System strengths

- ▶ Real web integration
- ▶ Real deployment path
- ▶ Real test evidence
- ▶ Extensible architecture

Table of Contents

- 1 Problem and Requirements
- 2 Database Design
- 3 SQL Implementation
- 4 Web Integration and Demo
- 5 Deployment Extension
- 6 Testing and Evaluation
- 7 Conclusion**

Limitations

- ▶ Some edge cases and delete flows are not fully covered by tests.
- ▶ Not every SQL object has a dedicated direct database test yet.
- ▶ Kubernetes and monitoring are demonstration-level extensions.
- ▶ The system does not yet support native mobile usage.
- ▶ Real-time bank synchronization is outside the current scope.

Future Development

Functional growth

- ▶ Recurring transactions
- ▶ Mobile application
- ▶ Real-time notifications

Technical growth

- ▶ Wider SQL object tests
- ▶ Stronger backup and restore
- ▶ Production deployment hardening

Final Conclusion

DBMS contribution

NUFI demonstrates a complete relational database design for personal finance management, implemented in MySQL with constraints, indexes, views, functions, procedures, and triggers.

System contribution

The database is integrated with Django, demonstrated through a working web application, tested on real MySQL, and extended with deployment and operational components.

Thank You

Thank You
Questions and Answers